

Introduction to Multimodal Learning: Problem Set 2

Due: 4.20

Please submit a zip file containing the following:

- An implemented version of `HW2.ipynb` that contains your implementation and output results.
- A `result` folder that saves your visualization results (`.gif` and figures).
- A `report.pdf` file briefly summarizing your results and observations. It should include result figures and answers to all questions.

It is recommended to run the notebook on Google Colab (with GPU), as CPU training can be slow. Each training run takes roughly 1–2 minutes.

1 Denoising Diffusion Probabilistic Models

Ring distribution is a typical multi-modal distribution. To evaluate DDPM's fitting capability, we choose a ring as the toy dataset.

Implementation Requirements: In the DDPM class of `HW2.ipynb`, implement the following three functions:

1. `forward_process`: Sample x_t from $q(x_t | x_0)$. Recall the forward process:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

2. `loss_fn`: Compute the denoising training loss. The model $\epsilon_\theta(x_t, t)$ predicts the noise ϵ added during the forward process.

3. `reverse_process`: Implement the DDPM reverse sampling step:

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) + \sigma_t z, \quad z \sim \mathcal{N}(0, I),$$

$$\text{where } \sigma_t = \sqrt{\tilde{\beta}_t} = \sqrt{\frac{(1 - \bar{\alpha}_{t-1}) \beta_t}{1 - \bar{\alpha}_t}}.$$

No need to tune any hyperparameters. Save your `.gif` file in the `result` folder and include results in your report.

2 Conditional Flow Matching

Flow matching is a simulation-free framework for training continuous normalizing flows. Instead of learning a discrete-step denoising process (as in DDPM), flow matching learns a continuous velocity field $v_\theta(x, t)$ that transports a simple distribution (e.g., Gaussian noise) to the data distribution along a smooth path.

2.1 Conditional Velocity Field

Define the linear interpolation path between noise $x_0 \sim \mathcal{N}(0, I)$ and data $x_1 \sim q_{\text{data}}$:

$$x_t = (1 - t)x_0 + tx_1, \quad t \in [0, 1].$$

- (a) Derive the conditional velocity field $u_t(x_t | x_1) = \frac{dx_t}{dt}$.
- (b) Write down the Conditional Flow Matching (CFM) training objective:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t, x_0, x_1} [\|v_\theta(x_t, t) - u_t(x_t | x_1)\|^2].$$

Explain why this is easier to compute than the intractable marginal flow matching loss.

2.2 Training CFM on 8-Gaussians

In HW2.ipynb, we provide a 2D *8-Gaussians* dataset (8 Gaussian clusters arranged in a circle).

Implementation Requirements:

1. Implement the `cfm_loss` function: sample $t \sim \mathcal{U}[0, 1]$, compute x_t , and return the CFM loss.
2. Implement the `euler_ode_solve` function: starting from $x_0 \sim \mathcal{N}(0, I)$, integrate the learned velocity field using Euler’s method:

$$x_{t+\Delta t} = x_t + v_\theta(x_t, t) \Delta t.$$

3. Train the model and generate samples.

2.3 Analysis

1. Compare generation quality with different numbers of Euler steps ($S \in \{1, 5, 10, 50, 100\}$). Include figures in your report.
2. Visualize the learned velocity field $v_\theta(x, t)$ at several time steps (e.g., $t = 0, 0.25, 0.5, 0.75$). Comment on how the velocity field changes over time.

3 Mean Flow

Consider a trained velocity field $v_\theta(x, t)$ from Section 2. The probability flow ODE is:

$$\frac{dz_t}{dt} = v_\theta(z_t, t), \quad z_0 \sim \mathcal{N}(0, I), \quad t \in [0, 1].$$

Standard ODE-based generation requires many integration steps. *Mean Flow* (Geng et al., 2025) introduces the **time-averaged velocity** to enable efficient few-step or even one-step generation.

3.1 MeanFlow Identity

Define the **mean velocity** from time 0 to time t along an ODE trajectory as:

$$\bar{u}(z_t, t) = \frac{z_t - z_0}{t}, \quad t > 0.$$

This is the average velocity over the interval $[0, t]$.

- (a) Derive the **MeanFlow Identity**:

$$\bar{u}(z_t, t) = v(z_t, t) - t \frac{d\bar{u}(z_t, t)}{dt}$$

Hint: Start from $z_t = z_0 + \int_0^t v(z_s, s) ds$. Differentiate $\bar{u} = (z_t - z_0)/t$ with respect to t along the ODE trajectory.

- (b) Show that $\lim_{t \rightarrow 0^+} \bar{u}(z_t, t) = v(z_0, 0)$.

Hint: Apply L'Hôpital's rule to $(z_t - z_0)/t$.

- (c) At $t = 1$, show that $\bar{u}(z_1, 1) = z_1 - z_0$. Explain why this means a perfect mean velocity predictor enables **one-step generation**:

$$z_1 = z_0 + \bar{u}(z_1, 1).$$

However, this requires knowing z_1 (the endpoint) as input to compute $\bar{u}$. Discuss why this circular dependency makes direct one-step generation non-trivial, and how one might work around it (e.g., training a displacement network $D_\theta(z_0) \approx z_1 - z_0$).

3.2 Verification and One-Step Generation

Using the trained flow matching model and the `euler_ode_solve` function from Section 2:

1. **Numerical verification:** Generate ODE trajectories (code provided). For several trajectories, compute and plot:
 - The instantaneous velocity $v(z_t, t)$ from the trained model.
 - The mean velocity $\bar{u}(z_t, t) = (z_t - z_0)/t$.
 - The quantity $v(z_t, t) - t \cdot d\bar{u}/dt$ (numerical derivative).

Verify that $\bar{u} \approx v - t \cdot d\bar{u}/dt$ holds along the trajectory.

2. **One-step generation:** We train a displacement network $D_\theta(z_0) \approx z_1 - z_0$ using ODE trajectory data (training code provided). Compare:
 - 100-step ODE generation (from Section 2).
 - 1-step mean flow generation: $z_1 = z_0 + D_\theta(z_0)$.

Include figures in your report and discuss the quality difference.

References

- Ho J, Jain A, Abbeel P. *Denoising diffusion probabilistic models*. NeurIPS, 2020.
- Lipman Y, Chen R T Q, Ben-Hamu H, et al. *Flow matching for generative modeling*. ICLR, 2023.
- Geng Z, Pokle A, Luo W, et al. *Mean Flows for one-step generative modeling*. arXiv preprint arXiv:2505.13447, 2025.
- MIT 6.S978: Deep Generative Models. <https://mit-6s978.github.io/schedule.html>

Please submit your code and results.